

Assignment Report

Contents

EXPERIMENT NO. 1	1
TITLE: Spring Boot Fundamentals - Project Setup, Bean Configuration and Logging	1
Aim	1
Objectives	1
Prerequisites	1
Theory	1
Architecture / Working Diagram	2
Scenario Description	2
Project Structure	2
Files Created	2
Implementation	2
Code Explanation	5
Execution Steps	5
Sample Output	5
Screenshots	6
Advantages	6
Limitations	6
Result	6
EXPERIMENT NO. 2	7
TITLE: RESTful Web Service using GET Method and MockMvc Testing	7
Aim	7
Objectives	7
Prerequisites	7
Theory	7
Architecture / Working Diagram	7
Scenario Description	8
Project Structure	8
Files Created	8
Implementation	8
Code Explanation	9
Sample Request	10
Sample Response	10
Execution Steps	10
Screenshots	10
Advantages	10
Limitations	10
Result	11

EXPERIMENT NO. 3 **12**

TITLE: RESTful Web Services for Employee and Department using Spring XML

Configuration	12
Aim	12
Objectives	12
Prerequisites	12
Theory	12
Architecture / Working Diagram	13
Scenario Description	13
Project Structure	13
Files Created	13
Implementation	13
Code Explanation	19
Sample Request	20
Sample Response	20
Execution Steps	20
Screenshots	20
Advantages	20
Limitations	21
Result	21

EXPERIMENT NO. 4 **22**

TITLE: RESTful Web Service using POST/PUT/DELETE Method with Input Validation

tion	22
Aim	22
Objectives	22
Prerequisites	22
Theory	22
Architecture / Working Diagram	23
Scenario Description	23
Project Structure	23
Files Created	23
Implementation	23
Code Explanation	28
Sample Request	28
Sample Response	28
Sample Request (validation failure)	28
Sample Response (validation failure)	29
Execution Steps	29
Screenshots	29
Advantages	29
Limitations	29
Result	29

EXPERIMENT NO. 5 **30**

TITLE: Securing RESTful Web Services using JWT Authentication with Spring Security

curity	30
Aim	30
Objectives	30
Prerequisites	30
Theory	30
Architecture / Working Diagram	31

Scenario Description	31
Project Structure	31
Files Created	31
Implementation	32
Code Explanation	38
Sample Request (Login)	38
Sample Response (Login)	38
Sample Request (Authenticated call using the JWT)	38
Sample Response (Unauthorized — missing/invalid token)	39
Execution Steps	39
Screenshots	39
Advantages	39
Limitations	39
Result	39
FINAL COMBINED PROJECT	41
Complete Final Project Structure	41
Consolidated application.properties	42
Consolidated pom.xml Dependencies	42
Overall Result	43

LAKSHMI NARAIN COLLEGE OF TECHNOLOGY

Department of Computer Science & Engineering

ASSIGNMENT REPORT

Spring Boot REST Web Services and JWT Security

Submitted By	VARUN JUNEJA
College	LNCT
Enrollment Number	0103CS231449
Superset ID	7809995
Email	varunjuneja1905@gmail.com
Academic Session	2025 - 2026

EXPERIMENT NO. 1

TITLE: Spring Boot Fundamentals - Project Setup, Bean Configuration and Logging

Aim

To create a Spring Boot web application using Spring Initializr, to load a bean from a Spring XML configuration file, and to incorporate logging in the application.

Objectives

1. To demonstrate creation of a Spring Boot application using Spring Initializr.
2. To explain the need and benefits of Spring Boot over plain Spring.
3. To demonstrate loading a bean from a Spring XML configuration file.
4. To demonstrate inclusion of logging in a Spring Boot application.

Prerequisites

- Java 17
- Spring Boot 3.x
- Maven
- VS Code / Eclipse / IntelliJ IDEA

Theory

Spring Boot is a framework built on top of the Spring Framework that simplifies the creation of stand-alone, production-ready Spring applications. It removes the need for extensive XML configuration by providing sensible defaults, auto-configuration, and an embedded server such as Tomcat. A developer can build and run a fully working web application without deploying it to an external server.

Spring Boot applications are usually generated using Spring Initializr, a web-based tool that lets a developer choose the build tool, language, Spring Boot version and dependencies, and then download a ready-to-use project skeleton. The generated project contains a main class annotated with `@SpringBootApplication`, which combines three annotations: `@Configuration`, `@EnableAutoConfiguration` and `@ComponentScan`. Calling `SpringApplication.run()` inside the `main()` method starts the embedded server and initializes the Spring IoC (Inversion of Control) container.

Even though Spring Boot reduces the need for XML, Spring still supports defining beans using an XML configuration file. A bean is any object that is managed by the Spring container. Beans can be defined with an id, a class, and constructor or setter arguments. `ClassPathXmlApplicationContext` is used to load such an XML file, and `context.getBean()` is used to retrieve the configured object. This is useful for centralizing configuration of common objects, such as a `SimpleDateFormat` instance, so the same configuration does not need to be repeated across the application.

Logging is essential to trace the flow of execution and to debug issues in production. Spring Boot uses SLF4J with Logback as the default logging framework. Logging levels (trace, debug, info, warn, error) can be configured per package in `application.properties`, allowing fine-grained control over how much detail is printed to the console.

Architecture / Working Diagram

```
Spring Initializr
  ↓
Generated Maven Project
  ↓
SpringApplication.run()
  ↓
Spring IoC Container (ApplicationContext)
  ↓
Beans (Java Config / XML Config)
  ↓
Application Logic + Logging (SLF4J)
```

Scenario Description

A new Spring Boot project named spring-learn needs to be created. A SimpleDateFormat bean with the pattern dd/MM/yyyy is required in multiple places of the application. Instead of creating it repeatedly, it is defined once in a Spring XML configuration file and retrieved wherever required. Logging is added so that the start of the application and the result of the date parsing can be traced in the console.

Project Structure

```
spring-learn
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── cognizant
│   │   │   │   │   ├── springlearn
│   │   │   │   │   │   └── SpringLearnApplication.java
│   │   └── resources
│   │       ├── application.properties
│   │       └── date-format.xml
└── pom.xml
```

Files Created

- SpringLearnApplication.java
- date-format.xml
- application.properties
- pom.xml

Implementation

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
```

```

    https://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.5</version>
  <relativePath/>
</parent>

<groupId>com.cognizant</groupId>
<artifactId>spring-learn</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>spring-learn</name>
<description>Spring Boot REST hands-on project</description>

<properties>
  <java.version>17</java.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

date-format.xml (src/main/resources)

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="dateFormat" class="java.text.SimpleDateFormat">
        <constructor-arg value="dd/MM/yyyy" />
    </bean>

</beans>
```

application.properties

```
server.port=8083

logging.level.org.springframework=info
logging.level.com.cognizant.springlearn=debug
logging.pattern.console=%d{yyMMdd} | %d{HH:mm:ss.SSS} | %-20.20thread | %5p | %-25.25logger{25} | %25M | %m
```

SpringLearnApplication.java

```
package com.cognizant.springlearn;

import java.text.SimpleDateFormat;
import java.util.Date;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

@SpringBootApplication
public class SpringLearnApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(SpringLearnApplication.class);

    public static void main(String[] args) {
        LOGGER.info("Starting SpringLearnApplication - main() method invoked");
        SpringApplication.run(SpringLearnApplication.class, args);
        displayDate();
    }

    public static void displayDate() {
        LOGGER.debug("displayDate() method invoked");

        ApplicationContext context =
```

```

        new ClassPathXmlApplicationContext("date-format.xml");

        SimpleDateFormat format =
            context.getBean("dateFormat", SimpleDateFormat.class);

        try {
            Date date = format.parse("31/12/2018");
            LOGGER.info("Parsed date: {}", date);
            System.out.println(date);
        } catch (Exception e) {
            LOGGER.error("Error parsing date", e);
        }
    }
}

```

Code Explanation

SpringLearnApplication.java — the `main()` method logs a start message, launches the embedded Tomcat server through `SpringApplication.run()`, and then calls `displayDate()`. This method loads `date-format.xml` using `ClassPathXmlApplicationContext`, retrieves the `date-Format` bean by name and type using `getBean()`, and uses it to parse a hard-coded date string. Every important step is logged using SLF4J instead of `System.out.println`, in line with production-quality logging practice.

date-format.xml — defines a single bean of type `SimpleDateFormat`, constructed with the pattern `dd/MM/yyyy` passed as a constructor argument. Defining it once means every part of the application that needs this date format can fetch the same configured instance from the Spring container instead of re-instantiating it.

application.properties — sets the server port to 8083 and configures per-package logging levels: `info` for framework classes and `debug` for the `com.cognizant.springlearn` package, together with a custom console log pattern.

Execution Steps

1. Open <https://start.spring.io> and generate a project with Group `com.cognizant`, Artifact `spring-learn`, and dependency `Spring Web`.
2. Extract the downloaded zip and open it in VS Code.
3. Add the `date-format.xml` file under `src/main/resources`.
4. Add the logging configuration to `application.properties`.
5. Implement the `displayDate()` method in `SpringLearnApplication.java`.
6. Run `mvn clean package` from the terminal to build the project.
7. Run the application using `mvn spring-boot:run` or by running the main class.
8. Observe the console output for the start log and the parsed date.
9. Confirm no errors are shown and the embedded Tomcat server starts on port 8083.

Sample Output

```

260706|10:12:03.221|main          |INFO |SpringLearnApplication |main|Starting Sprin
260706|10:12:04.897|main          |DEBUG|SpringLearnApplication |displayDate|display
260706|10:12:04.911|main          |INFO |SpringLearnApplication |displayDate|Parsed
Mon Dec 31 00:00:00 IST 2018

```

Screenshots

Insert VS Code Project Structure Screenshot Here

Insert Console Startup Log Screenshot Here

Insert Parsed Date Output Screenshot Here

Advantages

1. Spring Boot removes boilerplate configuration and provides an embedded server.
2. XML-based bean definition centralizes configuration and avoids duplication.
3. SLF4J logging gives fine control over log levels per package.

Limitations

1. Mixing XML configuration with Java/annotation configuration can reduce consistency.
2. Excessive logging at debug level can clutter the console in production.

Result

A Spring Boot project was successfully created, a SimpleDateFormat bean was loaded from an XML configuration file, and logging was successfully incorporated into the application.

EXPERIMENT NO. 2

TITLE: RESTful Web Service using GET Method and Mock-Mvc Testing

Aim

To understand the structure of an HTTP request and response, and to implement and test a RESTful “Hello World” web service using the GET method in Spring Boot.

Objectives

1. To explain HTTP Request and Response format in detail.
2. To explain the need and benefits of RESTful web services.
3. To implement a RESTful web service using the GET method.
4. To perform end-to-end testing of the RESTful web service using MockMvc.

Prerequisites

- Java 17
- Spring Boot 3.x
- Maven
- Postman / Browser

Theory

HTTP (HyperText Transfer Protocol) is the foundation of data exchange on the web. An HTTP request contains a method (GET, POST, PUT, DELETE), a resource path, the HTTP version, and headers such as Host, User-Agent and Accept-Language. An HTTP response contains a status line with a status code (such as 200 OK or 404 Not Found), response headers such as Content-Type, and a response body containing the actual data.

REST (REpresentational State Transfer) is an architectural style for designing web services. RESTful services are lightweight, stateless, and use standard HTTP verbs to operate on resources. Each resource is identified by a URL, and the same resource can support multiple HTTP methods depending on the operation required. REST services are maintainable, scalable, and technology-independent, since any client capable of making an HTTP call can consume them.

In Spring Boot, a REST endpoint is implemented using the `@RestController` annotation, which combines `@Controller` and `@ResponseBody`, so that the return value of a method is automatically written into the HTTP response body (typically serialized as JSON). The `@GetMapping` annotation maps a specific URL to a Java method that responds only to GET requests.

To verify a REST endpoint without starting a browser or Postman, Spring provides MockMvc, a testing utility that simulates HTTP requests inside a JUnit test. Using `@AutoConfigureMockMvc`, a test class can call `mockMvc.perform(get(...))` and assert the response using `andExpect(status().isOk())` or `jsonPath()` matchers. This allows REST APIs to be verified automatically as part of the build, both in the IDE and from the command line using Maven.

Architecture / Working Diagram

```
Client (Browser / Postman / MockMvc)
  ↓ GET /hello
DispatcherServlet
  ↓
HelloController (@RestController)
  ↓
sayHello() method
  ↓
Response Body: "Hello World!!"
```

Scenario Description

A REST endpoint needs to be added to the spring-learn project that returns the text Hello World!! when accessed using the GET method at the URL /hello. The endpoint must be verified from both the browser and Postman, and an automated MockMvc test must be written to confirm the response status and body.

Project Structure

```
spring-learn
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── cognizant
│   │   │   │   │   ├── springlearn
│   │   │   │   │   │   ├── SpringLearnApplication.java
│   │   │   │   │   │   └── controller
│   │   │   │   │   │       └── HelloController.java
│   │   └── test
│   │       ├── java
│   │       │   ├── com
│   │       │   │   ├── cognizant
│   │       │   │   │   ├── springlearn
│   │       │   │   │   │   ├── controller
│   │       │   │   │   │       └── HelloControllerTest.java
```

Files Created

- HelloController.java
- HelloControllerTest.java

Implementation

HelloController.java

```
package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
```

```

import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(HelloController.class);

    @GetMapping("/hello")
    public String sayHello() {
        LOGGER.info("Start - sayHello()");
        String response = "Hello World!!";
        LOGGER.info("End - sayHello() - response: {}", response);
        return response;
    }
}

```

HelloControllerTest.java

```

package com.cognizant.springlearn.controller;

import static org.hamcrest.Matchers.equalTo;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.content;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.web.servlet.MockMvc;

@SpringBootTest
@AutoConfigureMockMvc
class HelloControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void sayHello_returnsHelloWorldMessage() throws Exception {
        mockMvc.perform(get("/hello"))
            .andExpect(status().isOk())
            .andExpect(content().string(equalTo("Hello World!!")));
    }
}

```

Code Explanation

HelloController.java — annotated with `@RestController` so that the returned `String` is written directly as the response body. `@GetMapping("/hello")` maps GET requests on `/hello` to the `sayHello()` method. Start and end log statements are included as required, using SLF4J

at info level.

HelloControllerTest.java — uses `@SpringBootTest` with `@AutoConfigureMockMvc` to boot the full application context and inject a `MockMvc` instance. The test performs a GET request on `/hello`, asserts the HTTP status is 200 OK using `status().isOk()`, and asserts the response body exactly equals `Hello World!!`.

Sample Request

```
GET /hello HTTP/1.1
Host: localhost:8083
```

Sample Response

```
HTTP/1.1 200
Content-Type: text/plain;charset=UTF-8
Content-Length: 13

Hello World!!
```

Execution Steps

1. Create package `com.cognizant.springlearn.controller`.
2. Create `HelloController.java` with a `@GetMapping("/hello")` method.
3. Add start and end log statements inside `sayHello()`.
4. Start the application on port 8083.
5. Open `http://localhost:8083/hello` in Chrome and verify the response.
6. Open the same URL in Postman and verify headers and body.
7. Open Chrome DevTools → Network tab and inspect request/response headers.
8. Write `HelloControllerTest.java` using `MockMvc`.
9. Run the test from the IDE and confirm it passes.
10. Run `mvn test` from the command line and confirm the test passes.

Screenshots

Insert Chrome Browser Output Screenshot Here

Insert Postman Response and Headers Screenshot Here

Insert MockMvc Test Passing Screenshot Here

Advantages

1. REST GET endpoints are simple to implement and test.
2. `MockMvc` allows automated verification without a running server.
3. Testing can be run both from the IDE and Maven command line.

Limitations

1. GET requests should not be used to modify server state.
2. `MockMvc` tests do not cover real network-level issues (DNS, TLS, proxies).

Result

A RESTful “Hello World” web service using the GET method was successfully implemented, verified from the browser and Postman, and validated using an automated MockMvc test.

EXPERIMENT NO. 3

TITLE: RESTful Web Services for Employee and Department using Spring XML Configuration

Aim

To design and implement RESTful GET web services that return Employee and Department lists, where the underlying data is defined using Spring XML configuration, following the Controller → Service → DAO layered architecture.

Objectives

1. To demonstrate a layered REST web service architecture with Controller, Service and DAO.
2. To create static list data using Spring XML configuration.
3. To create a REST service that reads data from XML configuration and returns it as JSON.
4. To test the REST services using Postman.

Prerequisites

- Java 17
- Spring Boot 3.x
- Maven
- Postman

Theory

A well-designed REST application separates responsibilities into distinct layers. The **Controller** layer is responsible only for handling HTTP requests and responses and delegating business logic to the **Service** layer. The Service layer contains business logic and orchestrates calls to the **DAO (Data Access Object)** layer, which is responsible for retrieving and storing data. This separation of concerns makes the application easier to test, maintain, and extend.

In this experiment, instead of a database, static data is defined using Spring XML configuration. Each Employee and Department object is declared as a <bean> in an XML file, and lists of these beans are wired together using the <list> and <ref> tags. ClassPathXmlApplicationContext loads this file, and the DAO layer retrieves the pre-populated list using `getBean()`, storing it in a static variable for reuse.

The Service layer method is annotated with `@Transactional` even though no database transaction is involved here, following standard convention for service methods that may later be backed by real transactional resources. The Controller exposes two GET endpoints: `/employees`, which returns the full list of employees, and `/departments`, which returns the full list of departments. Both endpoints return Java objects directly from `@RestController` methods, and Spring automatically serializes them into JSON using the Jackson library bundled with `spring-boot-starter-web`.

This layered approach mirrors real-world enterprise applications, where the DAO layer can later be swapped from an XML-based static source to a JPA repository backed by a real database, without requiring any change to the Controller or Service layers.

Architecture / Working Diagram

```
Client (Postman)
  ↓ GET /employees , GET /departments
EmployeeController / DepartmentController
  ↓
EmployeeService / DepartmentService (@Service, @Transactional)
  ↓
EmployeeDao / DepartmentDao
  ↓
employee.xml / department.xml (Spring XML Configuration)
```

Scenario Description

An Angular front-end previously displayed a hard-coded employee list. This needs to be replaced by data served from a Spring Boot REST API. Employee and Department static data is defined in Spring XML configuration files, and two REST GET services are exposed: one that returns all employees, and one that returns all departments.

Project Structure

```
spring-learn
src main java com cognizant springlearn
    SpringLearnApplication.java
    controller/HelloController.java
    controller/EmployeeController.java
    controller/DepartmentController.java
    service/EmployeeService.java
    service/DepartmentService.java
    dao/EmployeeDao.java
    dao/DepartmentDao.java
    model/Employee.java
    model/Department.java
resources/employee.xml
resources/department.xml
```

Files Created

- Employee.java, Department.java
- employee.xml, department.xml
- EmployeeDao.java, DepartmentDao.java
- EmployeeService.java, DepartmentService.java
- EmployeeController.java, DepartmentController.java

Implementation

Employee.java

```
package com.cognizant.springlearn.model;

public class Employee {
```

```

private int id;
private String name;
private String skill;
private int departmentId;

public Employee() {}

public Employee(int id, String name, String skill, int departmentId) {
    this.id = id;
    this.name = name;
    this.skill = skill;
    this.departmentId = departmentId;
}

public int getId() { return id; }
public void setId(int id) { this.id = id; }
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public String getSkill() { return skill; }
public void setSkill(String skill) { this.skill = skill; }
public int getDepartmentId() { return departmentId; }
public void setDepartmentId(int departmentId) { this.departmentId = departmentId; }
}

```

Department.java

```

package com.cognizant.springlearn.model;

public class Department {

    private int id;
    private String name;

    public Department() {}

    public Department(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

```

employee.xml (src/main/resources)

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:util="http://www.springframework.org/schema/util"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

```

```

    xsi:schemaLocation="http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/util
    https://www.springframework.org/schema/util/spring-util.xsd">

    <bean id="employee1" class="com.cognizant.springlearn.model.Employee">
        <constructor-arg value="1" />
        <constructor-arg value="John" />
        <constructor-arg value="Java" />
        <constructor-arg value="1" />
    </bean>

    <bean id="employee2" class="com.cognizant.springlearn.model.Employee">
        <constructor-arg value="2" />
        <constructor-arg value="Sam" />
        <constructor-arg value="Angular" />
        <constructor-arg value="2" />
    </bean>

    <bean id="employee3" class="com.cognizant.springlearn.model.Employee">
        <constructor-arg value="3" />
        <constructor-arg value="Priya" />
        <constructor-arg value="Spring Boot" />
        <constructor-arg value="1" />
    </bean>

    <bean id="employee4" class="com.cognizant.springlearn.model.Employee">
        <constructor-arg value="4" />
        <constructor-arg value="Karan" />
        <constructor-arg value="SQL" />
        <constructor-arg value="3" />
    </bean>

    <util:list id="employeeList">
        <ref bean="employee1" />
        <ref bean="employee2" />
        <ref bean="employee3" />
        <ref bean="employee4" />
    </util:list>

</beans>

```

department.xml (src/main/resources)

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:util="http://www.springframework.org/schema/util"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/util
    https://www.springframework.org/schema/util/spring-util.xsd">

```

```

<bean id="department1" class="com.cognizant.springlearn.model.Department">
    <constructor-arg value="1" />
    <constructor-arg value="Engineering" />
</bean>

<bean id="department2" class="com.cognizant.springlearn.model.Department">
    <constructor-arg value="2" />
    <constructor-arg value="Quality Assurance" />
</bean>

<bean id="department3" class="com.cognizant.springlearn.model.Department">
    <constructor-arg value="3" />
    <constructor-arg value="Database Administration" />
</bean>

<util:list id="departmentList">
    <ref bean="department1" />
    <ref bean="department2" />
    <ref bean="department3" />
</util:list>

</beans>

```

EmployeeDao.java

```

package com.cognizant.springlearn.dao;

import java.util.ArrayList;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.stereotype.Repository;

import com.cognizant.springlearn.model.Employee;

@Repository
public class EmployeeDao {

    private static final Logger LOGGER = LoggerFactory.getLogger(EmployeeDao.class);

    @SuppressWarnings("unchecked")
    private static final ArrayList<Employee> EMPLOYEE_LIST;

    static {
        LOGGER.info("Loading employee list from employee.xml");
        ApplicationContext context = new ClassPathXmlApplicationContext("employee.xml");
        EMPLOYEE_LIST = (ArrayList<Employee>) context.getBean("employeeList");
    }

    public ArrayList<Employee> getAllEmployees() {
        LOGGER.info("Fetching all employees, count: {}", EMPLOYEE_LIST.size());
    }
}

```

```

        return EMPLOYEE_LIST;
    }
}

```

DepartmentDao.java

```

package com.cognizant.springlearn.dao;

import java.util.ArrayList;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.stereotype.Repository;

import com.cognizant.springlearn.model.Department;

@Repository
public class DepartmentDao {

    private static final Logger LOGGER = LoggerFactory.getLogger(DepartmentDao.class);

    @SuppressWarnings("unchecked")
    private static final ArrayList<Department> DEPARTMENT_LIST;

    static {
        LOGGER.info("Loading department list from department.xml");
        ApplicationContext context = new ClassPathXmlApplicationContext("department.xml");
        DEPARTMENT_LIST = (ArrayList<Department>) context.getBean("departmentList");
    }

    public ArrayList<Department> getAllDepartments() {
        LOGGER.info("Fetching all departments, count: {}", DEPARTMENT_LIST.size());
        return DEPARTMENT_LIST;
    }
}

```

EmployeeService.java

```

package com.cognizant.springlearn.service;

import java.util.ArrayList;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.cognizant.springlearn.dao.EmployeeDao;
import com.cognizant.springlearn.model.Employee;

@Service

```

```

public class EmployeeService {

    @Autowired
    private EmployeeDao employeeDao;

    @Transactional
    public ArrayList<Employee> getAllEmployees() {
        return employeeDao.getAllEmployees();
    }
}

```

DepartmentService.java

```

package com.cognizant.springlearn.service;

import java.util.ArrayList;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import com.cognizant.springlearn.dao.DepartmentDao;
import com.cognizant.springlearn.model.Department;

@Service
public class DepartmentService {

    @Autowired
    private DepartmentDao departmentDao;

    @Transactional
    public ArrayList<Department> getAllDepartments() {
        return departmentDao.getAllDepartments();
    }
}

```

EmployeeController.java

```

package com.cognizant.springlearn.controller;

import java.util.ArrayList;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import com.cognizant.springlearn.model.Employee;
import com.cognizant.springlearn.service.EmployeeService;

@RestController

```

```

public class EmployeeController {

    private static final Logger LOGGER = LoggerFactory.getLogger(EmployeeController.class);

    @Autowired
    private EmployeeService employeeService;

    @GetMapping("/employees")
    public ArrayList<Employee> getAllEmployees() {
        LOGGER.info("Start - getAllEmployees()");
        ArrayList<Employee> employees = employeeService.getAllEmployees();
        LOGGER.info("End - getAllEmployees() - returned {} records", employees.size());
        return employees;
    }
}

```

DepartmentController.java

```

package com.cognizant.springlearn.controller;

import java.util.ArrayList;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import com.cognizant.springlearn.model.Department;
import com.cognizant.springlearn.service.DepartmentService;

@RestController
public class DepartmentController {

    private static final Logger LOGGER = LoggerFactory.getLogger(DepartmentController.class);

    @Autowired
    private DepartmentService departmentService;

    @GetMapping("/departments")
    public ArrayList<Department> getAllDepartments() {
        LOGGER.info("Start - getAllDepartments()");
        ArrayList<Department> departments = departmentService.getAllDepartments();
        LOGGER.info("End - getAllDepartments() - returned {} records", departments.size());
        return departments;
    }
}

```

Code Explanation

EmployeeDao / DepartmentDao — a static block loads the respective XML configuration file once when the class is initialized, and stores the resulting list in a static ArrayList. `getAllEm-`

ployees() / getAllDepartments() simply return this cached list, avoiding repeated XML parsing on every request.

EmployeeService / DepartmentService — annotated with @Service and @Transactional, these classes delegate directly to the DAO layer. This layer is the natural place to add business rules in the future without affecting the Controller.

EmployeeController / DepartmentController — expose GET /employees and GET /departments respectively. Each method logs a start and end message and returns a Java ArrayList, which Spring automatically converts into a JSON array in the HTTP response.

Sample Request

```
GET /employees HTTP/1.1
Host: localhost:8083
```

Sample Response

```
[
  {"id":1,"name":"John","skill":"Java","departmentId":1},
  {"id":2,"name":"Sam","skill":"Angular","departmentId":2},
  {"id":3,"name":"Priya","skill":"Spring Boot","departmentId":1},
  {"id":4,"name":"Karan","skill":"SQL","departmentId":3}
]
```

Execution Steps

1. Create Employee.java and Department.java model classes.
2. Define employee.xml and department.xml under src/main/resources.
3. Create EmployeeDao and DepartmentDao, loading data from the respective XML file in a static block.
4. Create EmployeeService and DepartmentService, annotated @Service with a @Transactional method.
5. Create EmployeeController exposing GET /employees.
6. Create DepartmentController exposing GET /departments.
7. Start the application and open Postman.
8. Invoke GET http://localhost:8083/employees and verify the JSON array.
9. Invoke GET http://localhost:8083/departments and verify the JSON array.
10. Check the console logs to confirm both controllers were invoked.

Screenshots

Insert Postman Response for /employees Screenshot Here

Insert Postman Response for /departments Screenshot Here

Insert Console Log Output Screenshot Here

Advantages

1. Clear separation of Controller, Service and DAO layers improves maintainability.
2. XML-based static data allows quick prototyping without a database.
3. The DAO layer can later be replaced with a JPA repository with no change to the Controller.

Limitations

1. Static XML data is not suitable for production-scale or frequently changing data.
2. No persistence of updates — the data resets every time the application restarts.

Result

RESTful GET web services for Employee and Department were successfully implemented using a layered Controller-Service-DAO architecture, backed by Spring XML configuration, and verified using Postman.

EXPERIMENT NO. 4

TITLE: RESTful Web Service using POST/PUT/DELETE Method with Input Validation

Aim

To implement a complete RESTful CRUD web service for the Country resource using GET, POST, PUT and DELETE methods, with request payload validation and global exception handling.

Objectives

1. To explain the significance of HTTP method types (GET, POST, PUT, DELETE) in RESTful web services.
2. To follow RESTful URL naming guidelines for a resource.
3. To implement POST, PUT and DELETE endpoints using `@RequestBody`.
4. To validate input requests using Jakarta Validation annotations (`@NotNull`, `@NotBlank`, `@Size`).
5. To implement global exception handling using `@ControllerAdvice`.

Prerequisites

- Java 17
- Spring Boot 3.x
- Maven
- Postman / curl

Theory

RESTful web services use standard HTTP methods to represent CRUD (Create, Read, Update, Delete) operations on a resource. GET retrieves data about a resource, POST creates a new resource, PUT updates an existing resource, and DELETE removes a resource. The method type is only a convention — the application itself must implement the actual behaviour associated with each verb.

RESTful naming guidelines state that a resource should be named in plural form (`/countries`), a specific resource should be accessed by appending its identifier (`/countries/{code}`), and multi-word resource names should use hyphens rather than underscores. Following the same base URL for all methods on a resource, and defining it once at the class level using `@RequestMapping("/countries")`, keeps the API consistent and predictable.

To create or update a resource, the client sends data in the request body, typically as JSON. Spring maps this JSON to a Java object automatically when the controller method parameter is annotated with `@RequestBody`. However, client-supplied data cannot be trusted blindly — it must be validated. Jakarta Bean Validation (implemented by Hibernate Validator) provides annotations such as `@NotNull`, `@NotBlank`, `@Size`, `@Min` and `@Max` that can be placed directly on model fields. Adding `@Valid` before a `@RequestBody` parameter instructs Spring to automatically validate the incoming object and reject it with a 400 Bad Request if any constraint is violated.

Rather than letting each controller handle validation errors individually, a centralized `@ControllerAdvice` class with `@ExceptionHandler` methods can intercept validation exceptions (`MethodArgumentNotValidException`) application-wide and return a consistent, well-formatted

error response. This avoids duplicated error-handling code and ensures a uniform API contract for all failure cases, including malformed number inputs.

Architecture / Working Diagram

```
Client (Postman / curl)
  ↓ GET/POST/PUT/DELETE /countries
CountryController (@RequestMapping("/countries"))
  ↓
CountryService
  ↓
CountryDao (in-memory store)

Validation errors
  ↓
GlobalExceptionHandler (@ControllerAdvice)
  ↓
400 Bad Request (structured JSON error)
```

Scenario Description

A Country resource must be exposed as a full REST API. Clients should be able to retrieve all countries, retrieve one country by code, add a new country, update an existing country, and delete a country by code. All incoming country data must be validated so that a country code and name are always present and within defined length limits.

Project Structure

```
spring-learn
src main java com cognizant springlearn
    model/Country.java
    dao/CountryDao.java
    service/CountryService.java
    controller/CountryController.java
    exception/GlobalExceptionHandler.java
```

Files Created

- Country.java
- CountryDao.java
- CountryService.java
- CountryController.java
- GlobalExceptionHandler.java

Implementation

Country.java

```
package com.cognizant.springlearn.model;

import jakarta.validation.constraints.NotBlank;
```

```

import jakarta.validation.constraints.Size;

public class Country {

    @NotBlank(message = "Country code must not be blank")
    @Size(min = 2, max = 3, message = "Country code must be 2 to 3 characters")
    private String code;

    @NotBlank(message = "Country name must not be blank")
    @Size(min = 2, max = 60, message = "Country name must be 2 to 60 characters")
    private String name;

    public Country() {}

    public Country(String code, String name) {
        this.code = code;
        this.name = name;
    }

    public String getCode() { return code; }
    public void setCode(String code) { this.code = code; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

```

CountryDao.java

```

package com.cognizant.springlearn.dao;

import java.util.LinkedHashMap;
import java.util.Map;

import org.springframework.stereotype.Repository;

import com.cognizant.springlearn.model.Country;

@Repository
public class CountryDao {

    private final Map<String, Country> countryStore = new LinkedHashMap<>();

    public CountryDao() {
        countryStore.put("US", new Country("US", "United States"));
        countryStore.put("DE", new Country("DE", "Germany"));
        countryStore.put("IN", new Country("IN", "India"));
        countryStore.put("JP", new Country("JP", "Japan"));
    }

    public Iterable<Country> getAllCountries() {
        return countryStore.values();
    }

    public Country getCountry(String code) {

```

```

        return countryStore.get(code);
    }

    public Country addCountry(Country country) {
        countryStore.put(country.getCode(), country);
        return country;
    }

    public Country updateCountry(Country country) {
        countryStore.put(country.getCode(), country);
        return country;
    }

    public void deleteCountry(String code) {
        countryStore.remove(code);
    }
}

```

CountryService.java

```

package com.cognizant.springlearn.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.cognizant.springlearn.dao.CountryDao;
import com.cognizant.springlearn.model.Country;

@Service
public class CountryService {

    @Autowired
    private CountryDao countryDao;

    public Iterable<Country> getAllCountries() {
        return countryDao.getAllCountries();
    }

    public Country getCountry(String code) {
        return countryDao.getCountry(code);
    }

    public Country addCountry(Country country) {
        return countryDao.addCountry(country);
    }

    public Country updateCountry(Country country) {
        return countryDao.updateCountry(country);
    }

    public void deleteCountry(String code) {
        countryDao.deleteCountry(code);
    }
}

```

```
}
```

CountryController.java

```
package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.cognizant.springlearn.model.Country;
import com.cognizant.springlearn.service.CountryService;

import jakarta.validation.Valid;

@RestController
@RequestMapping("/countries")
public class CountryController {

    private static final Logger LOGGER = LoggerFactory.getLogger(CountryController.class);

    @Autowired
    private CountryService countryService;

    @GetMapping
    public Iterable<Country> getAllCountries() {
        LOGGER.info("Start - getAllCountries()");
        return countryService.getAllCountries();
    }

    @GetMapping("/{code}")
    public Country getCountry(@PathVariable String code) {
        LOGGER.info("Start - getCountry() - code: {}", code);
        return countryService.getCountry(code);
    }

    @PostMapping
    public Country addCountry(@Valid @RequestBody Country country) {
        LOGGER.info("Start - addCountry() - {}", country.getCode());
        Country saved = countryService.addCountry(country);
        LOGGER.info("End - addCountry()");
        return saved;
    }

    @PutMapping
```

```

public Country updateCountry(@Valid @RequestBody Country country) {
    LOGGER.info("Start - updateCountry() - {}", country.getCode());
    Country updated = countryService.updateCountry(country);
    LOGGER.info("End - updateCountry()");
    return updated;
}

@DeleteMapping("/{code}")
public void deleteCountry(@PathVariable String code) {
    LOGGER.info("Start - deleteCountry() - code: {}", code);
    countryService.deleteCountry(code);
    LOGGER.info("End - deleteCountry()");
}
}

```

GlobalExceptionHandler.java

```

package com.cognizant.springlearn.exception;

import java.util.LinkedHashMap;
import java.util.Map;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> handleValidationErrors(
        MethodArgumentNotValidException ex) {

        Map<String, String> errors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage()));

        return ResponseEntity.badRequest().body(errors);
    }

    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    public ResponseEntity<Map<String, String>> handleTypeMismatch(
        MethodArgumentTypeMismatchException ex) {

        Map<String, String> error = new LinkedHashMap<>();
        error.put("error", "Invalid value for parameter '" + ex.getName() + "'");

        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
    }
}

```

pom.xml (additional dependency)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Code Explanation

Country.java — a plain model class annotated with `@NotBlank` and `@Size` to enforce that both code and name are supplied and fall within sensible length limits.

CountryController.java — defines the base path `/countries` at the class level using `@RequestMapping`, and maps each CRUD operation to the correct HTTP verb: `@GetMapping`, `@GetMapping("/{code}")`, `@PostMapping`, `@PutMapping` and `@DeleteMapping("/{code}")`. The `@Valid` annotation on the `@RequestBody` parameter in `addCountry()` and `updateCountry()` triggers automatic validation before the method body executes.

GlobalExceptionHandler.java — annotated with `@RestControllerAdvice`, this class centrally handles `MethodArgumentNotValidException` (raised when `@Valid` fails) and `MethodArgumentTypeMismatchException` (raised for malformed path/number values), returning a structured JSON error body with HTTP 400 Bad Request instead of a generic server error page.

Sample Request

```
POST /countries HTTP/1.1
Host: localhost:8083
Content-Type: application/json
```

```
{
  "code": "FR",
  "name": "France"
}
```

Sample Response

```
{"code": "FR", "name": "France"}
```

Sample Request (validation failure)

```
POST /countries HTTP/1.1
Content-Type: application/json
```

```
{
  "code": "",
  "name": "F"
}
```

Sample Response (validation failure)

```
{
  "code": "Country code must not be blank",
  "name": "Country name must be 2 to 60 characters"
}
```

Execution Steps

1. Add the spring-boot-starter-validation dependency to pom.xml.
2. Create Country.java with @NotBlank and @Size constraints.
3. Create CountryDao.java with an in-memory map as the data store.
4. Create CountryService.java delegating to the DAO.
5. Create CountryController.java with @RequestMapping("/countries") and all five CRUD endpoints.
6. Add @Valid to the @RequestBody parameters in POST and PUT methods.
7. Create GlobalExceptionHandler.java with @RestControllerAdvice.
8. Start the application and test GET /countries in Postman.
9. Test POST /countries with both valid and invalid payloads and observe the 200/400 responses.
10. Test PUT and DELETE in Postman and confirm the in-memory store updates correctly.

Screenshots

Insert Postman GET /countries Screenshot Here

Insert Postman POST /countries Success Screenshot Here

Insert Postman POST /countries Validation Error Screenshot Here

Advantages

1. A consistent URL naming convention makes the API predictable for consumers.
2. Centralized validation prevents malformed data from reaching the service layer.
3. Global exception handling avoids duplicated error-handling logic across controllers.

Limitations

1. An in-memory map is not durable and resets when the application restarts.
2. Bean Validation covers structural correctness but not complex business rules (e.g. cross-field checks), which still require custom logic.

Result

A complete CRUD RESTful web service for the Country resource was implemented using GET, POST, PUT and DELETE methods, with input validation and centralized exception handling, and successfully verified using Postman.

EXPERIMENT NO. 5

TITLE: Securing RESTful Web Services using JWT Authentication with Spring Security

Aim

To secure the RESTful web services of the spring-learn project using Spring Security, and to implement stateless authentication and authorization using JSON Web Tokens (JWT).

Objectives

1. To secure web application endpoints using Spring Security.
2. To configure in-memory / custom authentication with roles.
3. To understand the structure and working of a JWT (Header, Payload, Signature).
4. To generate a JWT on successful login and set a token expiry.
5. To authorize subsequent requests using a custom Spring Security filter that validates the JWT.

Prerequisites

- Java 17
- Spring Boot 3.x
- Maven
- Spring Security
- JJWT library (io.jsonwebtoken)
- Postman

Theory

By default, a Spring Boot REST API has no authentication — any client can call any endpoint. Spring Security is the standard framework used to add authentication (verifying who the caller is) and authorization (verifying what the caller is allowed to do). Adding the `spring-boot-starter-security` dependency automatically secures every endpoint with HTTP Basic authentication using a randomly generated password, printed once in the console log at startup.

While Basic Authentication is simple, it requires the client to send the username and password with every request, and it relies on a server-side session (JSESSIONID) to track the authenticated user. This does not scale well for stateless REST APIs consumed by multiple types of clients (web, mobile, other services). JSON Web Token (JWT) solves this problem by allowing the server to issue a signed, self-contained token after a successful login. The client then sends this token in the `Authorization: Bearer <token>` header on every subsequent request, and the server verifies the token's signature and expiry without needing to look up a server-side session.

A JWT consists of three Base64-URL-encoded parts separated by dots: the **Header** (specifies the signing algorithm), the **Payload** (contains claims such as the username, roles, issued-at time and expiry time), and the **Signature** (computed using a secret key, which allows the server to detect if the token has been tampered with).

In Spring Security, JWT authentication is implemented by disabling the default session-based mechanism (`SessionCreationPolicy.STATELESS`), defining an `AuthenticationManager` and

UserDetailsService for verifying login credentials, and inserting a custom OncePerRequest-Filter before Spring Security's standard username/password filter. This custom filter reads the Authorization header on every request, validates the JWT using the shared secret key, and — if valid — populates the SecurityContext with the authenticated user's identity and roles so that role-based authorization (e.g. hasRole("USER")) works correctly for the rest of the request.

Architecture / Working Diagram



Scenario Description

All REST services in the spring-learn project (including /countries) must be secured. A user should authenticate once via a login endpoint using a username and password, receive a signed JWT in the response, and use that token in the Authorization header for all further requests. Access to /countries should be restricted to users holding the USER role.

Project Structure

```
spring-learn
src main java com cognizant springlearn
    security/SecurityConfig.java
    security/JwtUtil.java
    security/JwtAuthenticationFilter.java
    security/CustomUserDetailsService.java
    controller/AuthController.java
    model/AuthRequest.java
    model/AuthResponse.java
```

Files Created

- SecurityConfig.java
- JwtUtil.java
- JwtAuthenticationFilter.java
- CustomUserDetailsService.java
- AuthController.java
- AuthRequest.java, AuthResponse.java

Implementation

pom.xml (additional dependencies)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.5</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.5</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.5</version>
  <scope>runtime</scope>
</dependency>
```

application.properties (additional properties)

```
jwt.secret=ThisIsADemoSecretKeyForJwtSigningUsedInAcademicAssignmentOnly
jwt.expiration.ms=3600000
```

AuthRequest.java

```
package com.cognizant.springlearn.model;

public class AuthRequest {

    private String username;
    private String password;

    public AuthRequest() {}

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

AuthResponse.java

```
package com.cognizant.springlearn.model;

public class AuthResponse {

    private String token;

    public AuthResponse(String token) {
        this.token = token;
    }

    public String getToken() { return token; }
    public void setToken(String token) { this.token = token; }
}
```

CustomUserDetailsService.java

```
package com.cognizant.springlearn.security;

import java.util.List;

import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final PasswordEncoder passwordEncoder;

    public CustomUserDetailsService(PasswordEncoder passwordEncoder) {
        this.passwordEncoder = passwordEncoder;
    }

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        if ("admin".equals(username)) {
            return new User("admin", passwordEncoder.encode("pwd"),
                List.of(new SimpleGrantedAuthority("ROLE_ADMIN")));
        }
        if ("user".equals(username)) {
            return new User("user", passwordEncoder.encode("pwd"),
                List.of(new SimpleGrantedAuthority("ROLE_USER")));
        }
        throw new UsernameNotFoundException("User not found: " + username);
    }
}
```

JwtUtil.java

```
package com.cognizant.springlearn.security;

import java.security.Key;
import java.util.Date;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;

@Component
public class JwtUtil {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration.ms}")
    private long expirationMs;

    private Key getSigningKey() {
        return Keys.hmacShaKeyFor(secret.getBytes());
    }

    public String generateToken(String username) {
        Date now = new Date();
        Date expiry = new Date(now.getTime() + expirationMs);

        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(now)
            .setExpiration(expiry)
            .signWith(getSigningKey(), SignatureAlgorithm.HS256)
            .compact();
    }

    public String extractUsername(String token) {
        return parseClaims(token).getSubject();
    }

    public boolean isTokenValid(String token) {
        try {
            Claims claims = parseClaims(token);
            return claims.getExpiration().after(new Date());
        } catch (Exception e) {
            return false;
        }
    }

    private Claims parseClaims(String token) {

```

```

        return Jwts.parserBuilder()
            .setSigningKey(getSigningKey())
            .build()
            .parseClaimsJws(token)
            .getBody();
    }
}

```

JwtAuthenticationFilter.java

```

package com.cognizant.springlearn.security;

import java.io.IOException;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Autowired
    private JwtUtil jwtUtil;

    @Autowired
    private CustomUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
        throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String token = authHeader.substring(7);

            if (jwtUtil.isTokenValid(token)) {
                String username = jwtUtil.extractUsername(token);
                UserDetails userDetails = userDetailsService.loadUserByUsername(username);

                UsernamePasswordAuthenticationToken authToken =
                    new UsernamePasswordAuthenticationToken(

```

```

        userDetails, null, userDetails.getAuthorities());
    authToken.setDetails(
        new WebAuthenticationDetailsSource().buildDetails(request));

    SecurityContextHolder.getContext().setAuthentication(authToken);
}

filterChain.doFilter(request, response);
}
}

```

SecurityConfig.java

```

package com.cognizant.springlearn.security;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.EnableWebSecurity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired
    private JwtAuthenticationFilter jwtAuthenticationFilter;

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(
        AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth

```

```

        .requestMatchers("/auth/login").permitAll()
        .requestMatchers("/hello", "/employees", "/departments").permitAll()
        .requestMatchers("/countries/**").hasRole("USER")
        .anyRequest().authenticated()
    .addFilterBefore(jwtAuthenticationFilter,
        UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
}

```

AuthController.java

```

package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.cognizant.springlearn.model.AuthRequest;
import com.cognizant.springlearn.model.AuthResponse;
import com.cognizant.springlearn.security.JwtUtil;

@RestController
@RequestMapping("/auth")
public class AuthController {

    private static final Logger LOGGER = LoggerFactory.getLogger(AuthController.class);

    @Autowired
    private AuthenticationManager authenticationManager;

    @Autowired
    private JwtUtil jwtUtil;

    @PostMapping("/login")
    public AuthResponse login(@RequestBody AuthRequest authRequest) {
        LOGGER.info("Start - login() - username: {}", authRequest.getUsername());

        authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                authRequest.getUsername(), authRequest.getPassword()));

        String token = jwtUtil.generateToken(authRequest.getUsername());
        LOGGER.info("End - login() - token generated");

        return new AuthResponse(token);
    }
}

```

Code Explanation

CustomUserDetailsService.java — provides two in-memory users, admin (role ADMIN) and user (role USER), with passwords encoded using BCryptPasswordEncoder. This is used by the AuthenticationManager to validate credentials during login.

JwtUtil.java — generates a signed JWT containing the username as the subject, an issued-at timestamp, and an expiry timestamp (1 hour by default, configured via `jwt.expiration.ms`). It also exposes methods to extract the username from a token and to check whether a token is still valid (correctly signed and not expired).

JwtAuthenticationFilter.java — extends `OncePerRequestFilter` so it runs exactly once per request. It reads the `Authorization` header, checks for the `Bearer` prefix, validates the token using `JwtUtil`, and if valid, manually populates Spring Security's `SecurityContextHolder` with an authenticated user and their roles — this is what allows `hasRole("USER")` checks to work without a server-side session.

SecurityConfig.java — disables CSRF (not needed for a stateless JSON API), sets the session policy to STATELESS, permits /auth/login and the earlier read-only endpoints without authentication, restricts /countries/** to users with role USER, and inserts the custom JWT filter before Spring Security's default username/password filter.

AuthController.java — exposes POST /auth/login. It delegates credential checking to Spring Security's AuthenticationManager (which throws an exception automatically on bad credentials), and on success generates and returns a JWT.

Sample Request (Login)

```
POST /auth/login HTTP/1.1
Content-Type: application/json
```

```
{
  "username": "user",
  "password": "pwd"
}
```

Sample Response (Login)

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlc2VyIiwiaWF0IjoxNzUxNzk2MDAwLCJleHAiOiJlE3NTE3OTk2MD"
}
```

Sample Request (Authenticated call using the JWT)

```
GET /countries HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlc2VyIiwiaWF0IjoxNzUxNzk2MDAwLCJleHAiOiE3
```

Sample Response (Unauthorized — missing/invalid token)

```
{
  "timestamp": "2026-07-06T10:15:00.000+00:00",
  "status": 401,
  "error": "Unauthorized",
  "message": "Full authentication is required to access this resource",
  "path": "/countries"
}
```

Execution Steps

1. Add spring-boot-starter-security and JWT dependencies to pom.xml.
2. Add jwt.secret and jwt.expiration.ms to application.properties.
3. Create CustomUserDetailsService with in-memory admin and user accounts.
4. Create JwtUtil with token generation and validation methods.
5. Create JwtAuthenticationFilter extending OncePerRequestFilter.
6. Create SecurityConfig defining the SecurityFilterChain, session policy and URL authorization rules.
7. Create AuthController exposing POST /auth/login.
8. Start the application and confirm GET /countries now returns 401 Unauthorized without a token.
9. Call POST /auth/login with valid credentials in Postman and copy the returned token.
10. Call GET /countries again with the header Authorization: Bearer <token> and confirm it now returns 200 OK.

Screenshots

Insert Postman 401 Unauthorized Screenshot Here

Insert Postman Login Response with JWT Token Screenshot Here

Insert Postman Authenticated /countries Request Screenshot Here

Advantages

1. Stateless authentication removes the need for server-side session storage, improving scalability.
2. The same token can be reused across multiple microservices sharing the same secret/verification mechanism.
3. Role-based access control integrates directly with Spring Security's authorization rules.

Limitations

1. Once issued, a JWT cannot easily be revoked before its expiry without maintaining a blacklist.
2. If the signing secret is leaked, an attacker can forge valid tokens, so secret management is critical.
3. Token size is larger than a simple session id, slightly increasing request header size.

Result

RESTful web services in the spring-learn project were successfully secured using Spring Security, and stateless JWT-based authentication and role-based authorization were imple-

mented and verified using Postman.

FINAL COMBINED PROJECT

Complete Final Project Structure

```
spring-learn
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   ├── cognizant
│   │   │   │   │   ├── springlearn
│   │   │   │   │   │   ├── SpringLearnApplication.java
│   │   │   │   │   │   ├── controller
│   │   │   │   │   │   │   ├── HelloController.java
│   │   │   │   │   │   │   ├── EmployeeController.java
│   │   │   │   │   │   │   ├── DepartmentController.java
│   │   │   │   │   │   │   ├── CountryController.java
│   │   │   │   │   │   │   ├── AuthController.java
│   │   │   │   │   │   ├── service
│   │   │   │   │   │   │   ├── EmployeeService.java
│   │   │   │   │   │   │   ├── DepartmentService.java
│   │   │   │   │   │   │   ├── CountryService.java
│   │   │   │   │   │   ├── dao
│   │   │   │   │   │   │   ├── EmployeeDao.java
│   │   │   │   │   │   │   ├── DepartmentDao.java
│   │   │   │   │   │   │   ├── CountryDao.java
│   │   │   │   │   │   ├── model
│   │   │   │   │   │   │   ├── Employee.java
│   │   │   │   │   │   │   ├── Department.java
│   │   │   │   │   │   │   ├── Country.java
│   │   │   │   │   │   │   ├── AuthRequest.java
│   │   │   │   │   │   │   ├── AuthResponse.java
│   │   │   │   │   │   ├── exception
│   │   │   │   │   │   │   ├── GlobalExceptionHandler.java
│   │   │   │   │   │   ├── security
│   │   │   │   │   │   │   ├── SecurityConfig.java
│   │   │   │   │   │   │   ├── JwtUtil.java
│   │   │   │   │   │   │   ├── JwtAuthenticationFilter.java
│   │   │   │   │   │   │   ├── CustomUserDetailsService.java
│   │   │   │   │   │   └──
│   │   │   │   │   └──
│   │   │   │   └──
│   │   │   └──
│   │   └── resources
│   │       ├── application.properties
│   │       ├── date-format.xml
│   │       ├── employee.xml
│   │       └── department.xml
│   └── test
│       ├── java
│       │   ├── com
│       │   │   ├── cognizant
│       │   │   │   ├── springlearn
│       │   │   │   │   ├── controller
│       │   │   │   │   │   ├── HelloControllerTest.java
```

Consolidated application.properties

```
server.port=8083
```

```
logging.level.org.springframework=info
```

```
logging.level.com.cognizant.springlearn=debug
```

```
logging.pattern.console=%d{yyMMdd} |%d{HH:mm:ss.SSS} |%-20.20thread| %5p |%-25.25logger{25} |%25M |%m
```

```
jwt.secret=ThisIsADemoSecretKeyForJwtSigningUsedInAcademicAssignmentOnly
```

```
jwt.expiration.ms=3600000
```

Consolidated pom.xml Dependencies

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.5</version>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.5</version>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.5</version>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

```
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>
```

Overall Result

Across the five experiments, the spring-learn Spring Boot project evolved from a basic application with XML bean loading and logging, into a fully layered, validated, and JWT-secured RESTful web service exposing Employee, Department and Country resources — demonstrating the complete lifecycle of building production-style REST APIs with Spring Boot.